Denoising Using Wiener Filters

Konstantinos Roumoglou

April 30, 2025

Task A

I. Add Noise

We begin by loading the clean signal y using the `audioread` command. Then we generate white Gaussian noise with a standard deviation of 0.01 using the `randn` function. We add this noise to the original signal to create the noisy signal x:

$$\text{noise} = 0.01 \cdot \texttt{randn}(N), \quad N = \texttt{length}(y)$$
$$x = y + \text{noise}$$

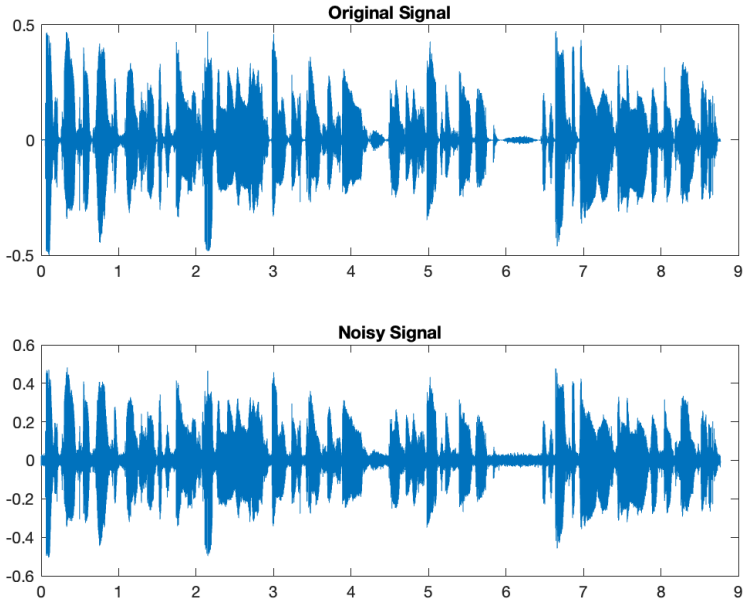

Figure 1: Signals

II. Wiener Filter

We assume $x(n)$ and $d(n)$ are wide-sense stationary (WSS) and aim to compute the filter coefficients $w(k),\ k = 0, 1, ..., p-1$ that minimize the mean square error (MSE):

$$\xi = \min_{w(k)} \mathbb{E}[|e(n)|^2] = \min_{w(k)} \mathbb{E}[|d(n) - \hat{d}(n)|^2] \tag{1}$$

$$= \frac{1}{n} \sum_{m=0}^{n} e^2[m] \tag{2}$$

To find the optimal weights:

$$\frac{\partial \xi}{\partial w^*(k)} = 0, \quad \text{for } k = 0, 1, ..., p-1 \tag{3}$$

The error $e(n)$ is:

$$e(n) = d(n) - \hat{d}(n) = d(n) - x(n) * w(n) \tag{4}$$

$$= d(n) - \sum_{l=0}^{p-1} w(l)x(n-l) \tag{5}$$

This leads to:

$$\mathbb{E}e(n)x^*(n-k) = 0 \tag{6}$$

Expanding:

$$r_{dx}(k) - \sum_{l=0}^{p-1} w(l)r_x(k-l) = 0 \tag{7}$$

This gives a system of equations:

$$\mathbf{R}xx \cdot \mathbf{w} = \mathbf{r}dx \tag{8}$$

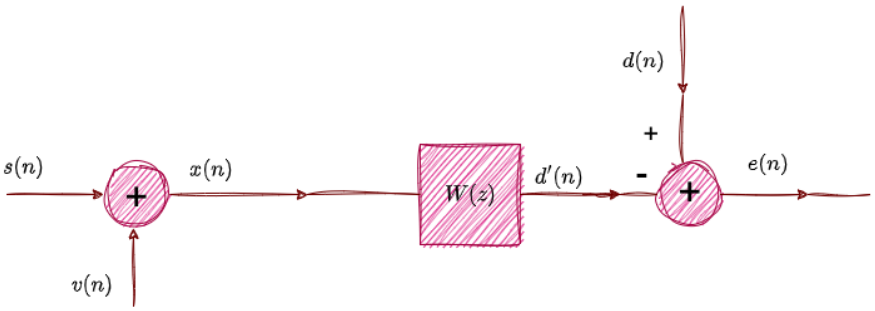

Figure 2: Wiener Filter

III. Signal-to-Noise Ratio (SNR)

SNR quantifies the strength of a desired signal relative to background noise:

$$\text{SNR}(a,b) = 10 \cdot \log_{10}\left(\frac{\text{Power}(a)}{\text{Power}(b)}\right), \quad \text{Power} = \frac{1}{N}\sum_{i=1}^{N} x(i)^2 \tag{9}$$

Results:

- Noisy signal: SNR = 19.39 dB
- Order 10: SNR = 19.72 dB
- Order 20: SNR = 19.73 dB
- Order 30: SNR = 19.74 dB

Task B

Instead of filtering the entire signal, we divide it into overlapping frames of 256 samples using a 50 % overlap. Each frame is filtered individually using the Wiener filter and then reconstructed. This approach significantly improves performance. We implement this approach with the help of *"frame_wind.m"* that divides the signal into frames and *"frame_recon.m"* that reconstructs the filtered signal.

Results:

- Order 10: SNR = 21.56 dB
- Order 20: SNR = 21.82 dB
- Order 30: SNR = 22.02 dB

Task C

We know that white noise has no correlation between its samples, except at point 0. In other words, it is sufficient to calculate the autocorrelation of the noise, which should be zero except at a single point. Below is the graph of the autocorrelation of white Gaussian noise.

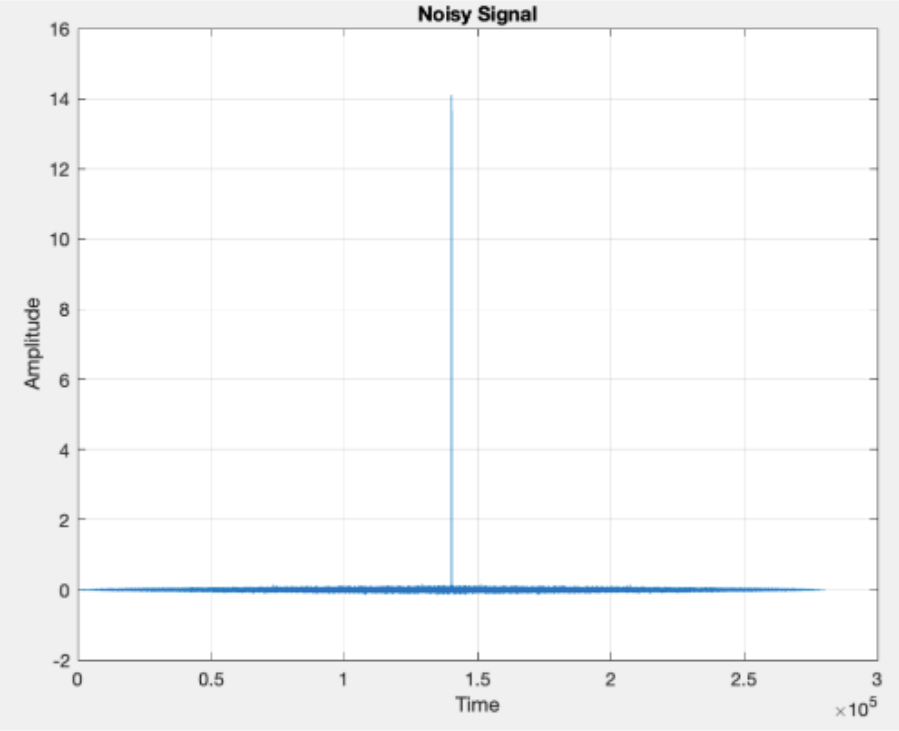

Figure 3: Noise Autocorrelation

Task D

We will repeat **Task A** without knowing the original signal and, therefore, without access to its autocorrelation. We will identify a portion of the noisy signal that contains silence — that is, only noise — and from there compute the autocorrelation of the noise. Clearly, this segment will be shorter than the original signal, but what we are interested in are the 10, 20, or 30 points around the center, depending on the order of the filter we will implement.

The noise is white gaussian and therefore uncorrelated.

noise(n) = v(n)

$$r_{yx}(k) = \mathbb{E}\left\{y(n)x^{\dagger}(n+k)\right\} = \mathbb{E}\left\{y(n)y^{\dagger}(n+k)\right\} + \mathbb{E}\left\{y(n)v^{\dagger}(n+k)\right\} \quad (*)$$

Since y and v are uncorrelated:

$$\mathbb{E}\left\{y(n)v^{\dagger}(n+k)\right\} = 0 \quad (*) \Rightarrow$$

$$r_{yx}(k) = r_y(k)$$

$$r_x(k) = \mathbb{E}\left\{x(n)x^{\dagger}(n+k)\right\} = \mathbb{E}\left\{y(n)y^{\dagger}(n+k)\right\} + \mathbb{E}\left\{v(n)v^{\dagger}(n+k)\right\} + \mathbb{E}\left\{y(n)v^{\dagger}(n+k)\right\} + \mathbb{E}\left\{v(n)y^{\dagger}(n+k)\right\} \Rightarrow$$

$$r_x(k) = r_y(k) + r_v(k)$$

So we implement a new function *"find_silence.m"* that finds a portion with silence(=noise) and we call the *"neWiener.m"* that implements the Wiener filter with some modifications,without knowing the original signal.

Results:

- Order 10: SNR = 21.56 dB
- Order 20: SNR = 21.82 dB
- Order 30: SNR = 22.02 dB

Task E

In this task we repeat **Part b** without having access to the original signal y, and therefore without its autocorrelation.

To achieve this , we divide the signal into 1095 frames of 256 each using *"frame_wind.m"* function.hen, we iterate over each frame (or column) and determine whether it contains speech or only noise using the detection mechanism implemented in the previous task. If a frame contains only noise, we use it to compute the autocorrelation of the noise, and consequently, estimate the autocorrelation of the original signal. This estimation is used as a fixed reference until a new noise-only frame is found. In each frame ,we apply the Wiener filter suing the *"neWiener.m"* function. Finally ,we reconstruct the denoised signal using the *"frame_recon.m"* function and evaluate the results. The entire process is implemented by the *"neWiener_frames.m"* function.

Part F

We use the structure from **Task B** to apply the Wiener filter for predicting the future values (e.g., 2,10 or 15 samples ahead). So we create the *"wiener_predict.m"* function which predicts the future samples of the signal based on the input provided.